SISTEMA FCT — PLAN DE QA

Guía Exhaustiva de Pruebas Funcionales y Seguridad

Código:	DOC-QA-FCT-001
Versión:	1.0.0 (Oficial)
Fecha:	Octubre 2026
Clasificación:	Uso Interno QA

1. PROPÓSITO Y ALCANCE

Este documento describe cómo probar el estado funcional del Sistema FCT desde la base de datos hasta la interfaz web. Incluye la preparación de un entorno aislado, las comprobaciones automatizadas existentes, recorridos manuales por rol, validación negativa, seguridad básica, concurrencia, interfaz y criterios para registrar resultados.

La prueba automatizada disponible hoy es `tests/run.php` : verifica algunos flujos de modelos y middleware contra MariaDB. Los recorridos de navegador, seguridad, compatibilidad entre navegadores, concurrencia y operación que aparecen en esta guía son procedimientos manuales; **no se afirma que exista una suite automatizada que los ejecute**.

> **ADVERTENCIA DE SEGURIDAD:** La guía es solo para testing. No usar datos personales reales ni ejecutar estas pruebas contra una base que se quiera conservar.

2. CONDICIONES DE SEGURIDAD ANTES DE EMPEZAR

1. Utiliza una base exclusiva de pruebas cuyo nombre empiece por `fct_test_` .
2. No ejecutes `tests/run.php` contra `sistema_fct` , ni contra una base de desarrollo con información útil: la prueba inserta y elimina registros.
3. Para pruebas manuales web, sirve también la misma base de pruebas. Una conexión CLI aislada no basta si Apache sigue conectado a otra base.
4. Usa solamente datos sintéticos, correos reservados como `example.test` y documentos PDF de prueba sin información personal.
5. El dump y las migraciones son para inicialización de una base vacía y su esquema no debe modificarse a mano durante una ejecución de pruebas funcionales. Si necesitas repetir toda la batería, elimina únicamente la base de pruebas y créala de nuevo.
6. No publiques una página local expuesta a Internet. Para pruebas web locales, usa `127.0.0.1` .
7. Al terminar, cierra las sesiones, detén el servidor de prueba y elimina la base y los ficheros temporales de prueba que ya no necesites.

3. REQUISITOS Y DATOS DE PRUEBA

- XAMPP con Apache, MariaDB y PHP CLI (extensiones `pdo_mysql` , `fileinfo` y `mbstring`).
- Navegador moderno (se recomiendan perfiles separados o ventana privada por cuenta).
- Cliente MariaDB (`mysql.exe`) y phpMyAdmin para comprobaciones SQL.
- Carpeta escribible `storage/uploads/convenios/` .
- Para cargar convenios: PDF inocuo de prueba < 10 MB, un archivo no-PDF renombrado a `.pdf` y un archivo > 10 MB.

El dump trae cinco cuentas demostrativas (contraseña por defecto: `password123`):

Rol	Correo Electrónico	Notas de Uso
Administrador	`admin@centro.com`	Gestión de usuarios y sistema
Coordinador	`coordinador@centro.com`	Revisión de convenios, ofertas y matching
Empresa	`empresa.tech@gmail.com`	Publicación de ofertas y gestión de sedes
Alumno (Menor)	`alumno.menor@alumno.com`	Requiere datos de tutor legal
Alumno (Mayor)	`alumno.mayor@alumno.com`	Registro estándar sin tutor

Nota: La cuenta Observador no viene en el dump; se crea desde el panel del Administrador.

> **Convención recomendada:** Añade un sufijo único (ej. `qa-20261008-01`) a cada correo, CIF, DNI/NIE, NSS y número de convenio. Anota los IDs de empresa, expediente y convenio en phpMyAdmin.

4. CREAR LA BASE AISLADA DESDE CERO

4.1 Crear esquema nuevo con phpMyAdmin

1. Inicia Apache y MySQL en XAMPP. Abre phpMyAdmin en la pestaña **Bases de datos**.
2. Crea `fct_test_manual` con `utf8mb4` y collation `utf8mb4_unicode_ci`.
3. Selecciona `fct_test_manual`, abre **Importar** y ejecuta `database/sistema_fct.sql`.
4. Comprueba la creación de las tablas: `usuarios`, `alumnos`, `coordinadores`, `direcciones_telefonos`, `empresas`, `empresa_especialidades_plazas`, `especialidades` y `tutores_legales`.
5. Importa la migración `database/migrations/001_renombrar_nombre_especialidad.sql`.
6. Importa a continuación `database/migrations/002_fct_workflows.sql`.
7. Verifica la adición de columnas/tablas: `activo`, `fecha_nacimiento`, `persona_contacto`, `empresa_sedes`, `convenios`, `alumno_disponibilidad`, `asignaciones_fct` y `auditoria`.
8. **Importante:** No importes `database/seeds/001_sample_data.sql` (contiene un `USE sistema_fct` que destruye el aislamiento).

4.2 Comprobar datos de demostración

En phpMyAdmin, ejecuta estas consultas tras la migración para validar el estado inicial:

```
SELECT DATABASE();
SHOW TABLES;
SHOW COLUMNS FROM especialidades;
SELECT id, email, rol FROM usuarios ORDER BY id;
SELECT id, nombre, apellidos, especialidad_id FROM alumnos ORDER BY id;
SELECT id, empresa_id, especialidad_id, estado, fecha_inicio, fecha_fin
FROM empresa_especialidades_plazas ORDER BY id;
```

Resultados esperados: La base actual es `fct_test_manual`. La tabla `especialidades` posee la columna `nombre_especialidad`. Existen 5 usuarios demo, 2 alumnos y 1 empresa. La oferta importada tiene estado `Pendiente`.

5. EJECUTAR LA PRUEBA AUTOMATIZADA EXISTENTE

Abre PowerShell en la raíz del repositorio y ejecuta el script de pruebas en entorno aislado:

```
$env:FCT_TEST_DB_NAME = "fct_test_manual"
$env:FCT_DB_NAME = $env:FCT_TEST_DB_NAME
& "C:\xampp\php\php.exe" tests\run.php
```

Salida esperada: `OK: migraciones, revisión de convenios/ofertas, matching, capacidad, cierre de prácticas y CSRF.`

5.1 Comprobar protección contra base mal configurada

Verifica que la suite rechaza nombres de base inseguros o discrepancias de entorno:

```
# Prueba 1: Rechazo de base sin prefijo de prueba
$env:FCT_TEST_DB_NAME = "sistema_fct"; $env:FCT_DB_NAME = "sistema_fct"
& "C:\xampp\php\php.exe" tests\run.php  # Debe fallar con código != 0

# Prueba 2: Discrepancia de variables
$env:FCT_TEST_DB_NAME = "fct_test_manual"; $env:FCT_DB_NAME = "fct_test_otra"
& "C:\xampp\php\php.exe" tests\run.php  # Debe fallar por descalce
```

5.2 Validar sintaxis de PHP (Lint)

```
$php = "C:\xampp\php\php.exe"; $failed = $false
Get-ChildItem app, config, public, routes, tests -Filter *.php -Recurse | ForEach-Object {
    & $php -l $_.FullName
    if ($LASTEXITCODE -ne 0) { $failed = $true }
}
if ($failed) { throw "Falló el lint de PHP." }
```

7. PRUEBAS DE ENRUTAMIENTO Y RESPUESTAS COMUNES

Ejecuta cada caso en el navegador o mediante peticiones HTTP estructuradas y registra los resultados:

ID	Acción	Procedimiento	Resultado Esperado
RT-01	Inicio	Abre `/` o `index.php`.	HTTP 200, título Sistema FCT y bienvenida.
RT-02	Acción desconocida	Abre `index.php?action=no-existe`.	Página 404; sin trazas de código PHP o SQL.
RT-03	Método incorrecto	POST a ruta GET (ej. `?action=ofertas`).	HTTP 405 con cabecera `Allow` correspondiente.
RT-04	Logout GET	Abre `?action=logout` vía GET.	HTTP 405; la sesión debe permanecer activa.
RT-05	Recurso privado	Sin sesión, abre `?action=usuarios`.	Redirección a login; no expone datos.
RT-06	Rol no autorizado	Alumno intenta abrir `?action=usuarios`.	HTTP 403 (Acceso Denegado sin vista admin).
RT-07	POST sin CSRF	POST a ruta protegida sin token `_csrf`.	HTTP 403; no modifica la base de datos.
RT-08	CSRF Inválido	Envía `_csrf=invalid` en formulario.	HTTP 403; operación rechazada.
RT-09	CSRF Válido	Envía formulario oficial con token válido.	Pasa validación CSRF y procesa negocio.

8. PRUEBAS DE AUTENTICACIÓN Y SESIÓN

8.1 Acceso correcto e incorrecto

1. Accede a `?action=login`. Prueba credenciales incorrectas: debe mostrar mensaje genérico sin indicar si el correo existe.
2. Prueba validación de formulario: correo vacío, formato inválido y contraseña en blanco.
3. Inicia sesión con cada perfil demo usando `password123`. Verifica redirecciones: Empresa va a publicación de ofertas; demás roles van al catálogo de ofertas.
4. Cierra sesión y presiona "Atrás" en el navegador: la página privada debe denegar el acceso y pedir login.
5. Inspecciona la cookie de sesión: debe contener las directivas `HttpOnly` y `SameSite=Lax` (y `Secure` bajo HTTPS).

8.2 Estado y regeneración de sesión

1. Anota el ID de cookie de sesión antes del login. Inicia sesión y verifica que el ID de cookie se regenera.
2. Desactiva la cuenta desde una sesión de Administrador. Intenta navegar con la cuenta desactivada: la siguiente petición debe invalidar la sesión y redirigir a login.

8.3 Límites conocidos

El sistema no implementa recuperación de contraseña, verificación por correo ni bloqueo por intentos fallidos. Registra estos puntos como **no disponibles**.

9. PRUEBAS DE REGISTRO DE ALUMNO

9.1 Alta válida de menor de edad (16-17 años)

1. Abre `?action=registro-alumno`. Rellena todos los campos con datos sintéticos.
2. Indica fecha de nacimiento equivalente a 16 o 17 años. Rellena obligatoriamente los datos del tutor legal.
3. Usa una contraseña de al menos 12 caracteres ASCII. Selecciona un ciclo de la lista.
4. Envía el formulario: debe redirigir al login confirmando el alta.
5. Verifica en DB las filas creadas e interrelacionadas en `usuarios`, `alumnos`, `direcciones_telefonos` y `tutores_legales`.

9.2 Alta válida de adulto (18+ años)

Repite el proceso con fecha de nacimiento de mayor de edad. Deja vacíos los campos del tutor legal. Se debe crear el expediente sin registro en `tutores_legales`.

9.3 Validaciones negativas del registro

Verifica el rechazo atómico ante: correos o DNI/NSS duplicados, contraseñas fuera de rango (menores a 12 o superiores al límite), campos requeridos ausentes, menores de 16 años, menores de 18 sin tutor, e inyección de arrays en lugar de

10. PRUEBAS DE REGISTRO DE EMPRESA Y SEDES

10.1 Registro válido de empresa

1. Abre `?action=registro-empresa`. Introduce correo, contraseña, razón social, CIF, persona de contacto y dirección sintéticos.
2. Confirma la creación de registros en `usuarios`, `empresas` y `direcciones_telefonos`.
3. Inicia sesión como Empresa y verifica el menú: Publicar oferta, Sedes y Convenios.

10.2 Validaciones negativas y gestión de sedes

- **Negativas:** Verifica rechazo por CIF duplicado, campos vacíos, arrays inyectados o sin token CSRF.
- **Sedes (`?action=sedes`):** Registra una nueva sede con nombre y dirección válidos. Comprueba su aislamiento: una Empresa B no debe ver las sedes creadas por la Empresa A.
- **Control de acceso:** Alumnos, Coordinadores u Observadores deben recibir HTTP 403 al intentar acceder a la ruta de sedes.

11. PRUEBAS DE CONVENIOS Y DOCUMENTOS PDF

11.1 Camino positivo de carga y revisión

1. Como Empresa, accede a `?action=convenios`. Carga un PDF válido (< 10 MB) con fechas coherentes y número único.
2. Verifica la creación del registro en estado `Pendiente` y el almacenamiento del archivo con nombre aleatorio único en `storage/uploads/convenios/` (nunca bajo `public/`).
3. Como Coordinador, abre `?action=revisar-convenios`. Descarga el PDF, verifica la integridad de su contenido y aprueba el convenio. Aprueba un segundo convenio y rechaza un tercero indicando motivo.
4. Confirma los campos de auditoría: `revisado_por`, `revisado_en`, `estado` y motivo de rechazo.
5. **Aislamiento de descarga:** Verifica que una Empresa distinta recibe HTTP 403 si intenta descargar el convenio de otra entidad.

11.2 Pruebas negativas del upload de convenios

Prueba los siguientes casos de fallo: archivo no adjuntado, archivos TXT/PNG renombrados a `.pdf`, PDF corrupto de 0 bytes, archivos > 10 MB, fecha de inicio posterior a la de fin, fecha de firma posterior al inicio, y sedes pertenecientes a otra empresa. El sistema no debe crear registros ni dejar archivos huérfanos en disco.

11.3 Acceso directo a archivos

Intenta acceder directamente vía URL del navegador a la ruta `/storage/uploads/convenios/fichero.pdf`. Debe devolver HTTP 403 Forbidden. La descarga legítima debe incluir cabeceras `Content-Type: application/pdf` y `Content-Disposition: attachment`.

12. PRUEBAS DE PUBLICACIÓN Y REVISIÓN DE OFERTAS

12.1 Publicación y aprobación

1. Como Empresa con convenio aprobado, registra una oferta (1-100 plazas, turno M/T/Indiferente, fechas dentro de la vigencia del convenio).
2. Verifica estado inicial `Pendiente`. Como Coordinador, accede a `?action=revisar-ofertas` y apruébala.
3. Accede de forma anónima a `?action=ofertas` y confirma que la oferta aparece en el catálogo público.

12.2 Validaciones negativas y visibilidad de catálogo

Verifica que no se puede publicar ni aprobar ofertas si el convenio está pendiente/rechazado o si las fechas de la oferta exceden el convenio. Ofertas pendientes, rechazadas o caducadas no deben aparecer en el catálogo público.

13. PRUEBAS DE DISPONIBILIDAD DEL ALUMNO

1. Inicia sesión como Alumno e ingresa a `?action=disponibilidad`.
2. Guarda un periodo futuro y turno. Guarda un segundo periodo: confirma que se actualiza el registro existente y no se crean filas duplicadas.
3. Prueba validaciones negativas: fecha inicio pasada, fin anterior al inicio, turnos alterados.
4. Verifica la seguridad: el POST no debe aceptar un `alumno_id` manipulado; la identidad debe obtenerse estrictamente de la sesión.

14. PRUEBAS DE MATCHING Y ASIGNACIÓN

14.1 Matriz de compatibilidad de turnos

En `?action=emparejar`, comprueba que el algoritmo respeta la matriz de coincidencia de turnos y especialidades:

Disponibilidad Alumno	Turno Oferta	Resultado Compatibilidad
Mañana (`M`)	Mañana (`M`)	**Compatible**
Tarde (`T`)	Tarde (`T`)	**Compatible**
Indiferente	Mañana o Tarde	**Compatible**
Mañana o Tarde	Indiferente	**Compatible**
Mañana (`M`)	Tarde (`T`)	**Incompatible**

14.2 Asignación dentro del cupo

1. Como Coordinador, asigna un alumno compatible a una oferta con 1 plaza indicando tutor de empresa.
2. Verifica el estado `Asignada`, el registro en auditoría y que el alumno deja de figurar como disponible para otras ofertas.
3. Intenta asignar un segundo alumno a la misma oferta: debe ser rechazado por cupo agotado.

14.3 Concurrencia en la asignación

> **Prueba de Estres de Concurrencia:** Prepara una oferta de 1 plaza y 2 candidatos. Abre 2 sesiones de Coordinador en navegadores independientes. Envía la asignación de ambos candidatos simultáneamente. Verifica que solo una solicitud prospera y la base de datos no excede el límite de plazas.

15. PRUEBAS DE CIERRE DE PRÁCTICAS

1. Como Coordinador, abre `?action=asignaciones` y ejecuta el cierre de una práctica activa.
2. Confirma la actualización a estado `Cerrada`, la marca de tiempo de cierre y el log de auditoría.
3. Intenta enviar un segundo POST de cierre sobre la misma asignación: debe ser denegado.
4. Roles no autorizados (Alumno, Empresa, Observador) deben recibir HTTP 403.

16. PRUEBAS DE GESTIÓN DE CUENTAS INTERNAS

1. Como Administrador, abre `?action=usuarios`. Verifica que no se muestran contraseñas ni hashes.
2. Crea usuarios con roles Coordinador, Observador y Administrador. Comprueba la creación de sus correspondientes perfiles.
3. **Reglas de autoprotección:** Intenta desactivar tu propia cuenta activa: debe ser rechazado. Deja un solo Administrador e intenta desactivarlo: el sistema debe impedir la operación.

17. PRUEBAS DE CONSULTAS Y VISIBILIDAD

1. Como Administrador, Coordinador u Observador, accede a `?action=consultas`.
2. Prueba los filtros por especialidad, estado de oferta y rangos de fechas. Comprueba el límite máximo de 500 resultados.
3. Inyección HTML: Introduce etiquetas `<script>` en los datos de prueba y verifica que la vista los renderiza estrictamente escapados como texto plano.

18. PRUEBAS DE SESIÓN POR MATRIZ DE ROLES

Verifica los permisos de acceso a las rutas según la siguiente matriz oficial de autorización:

Ruta / Acción	Admin	Coord.	Observ.	Empresa	Alumno	Anónimo
`ofertas`	Sí	Sí	Sí	Sí	Sí	Sí
`publicar-oferta` / `sedes` / `convenios`	No	No	No	Sí	No	No
`revisar-convenios` / `revisar-ofertas`	Sí	Sí	No	No	No	No
`disponibilidad` / `mis-asignaciones`	No	No	No	No	Sí	No
`emparejar`	Sí	Sí	No	No	No	No
`asignaciones` / `consultas`	Sí	Sí	Sí	No	No	No
`usuarios`	Sí	No	No	No	No	No
`descargar-convenio`	Sí	Sí	Sí	Propio	No	No

19. PRUEBAS DE SEGURIDAD WEB Y DATOS

19.1 Protección CSRF y sanitización

- Elimina o altera el campo `_csrf` en solicitudes POST: debe retornar HTTP 403 sin modificar datos.
- **XSS Output Encoding:** Introduce `<script>alert('test')</script>` en campos de texto. Al renderizarse en las vistas, debe aparecer neutro.
- **SQL Injection:** Introduce comillas y secuencias de escape (ej. `' OR '1'='1`) en login y buscadores. Debe ser procesado como literal sin alterar la lógica SQL.

19.2 Cabeceras de seguridad y cookies

Verifica las siguientes cabeceras en las respuestas HTTP:

```
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN / DENY
Referrer-Policy: strict-origin-when-cross-origin
```

20. PRUEBAS DE INTERFAZ Y ACCESIBILIDAD

1. Verifica la aplicación en navegadores Chromium y Firefox.
2. Prueba el diseño adaptativo en resoluciones de escritorio (1366px), tablet (768px) y móvil (375px).
3. Navega por los formularios utilizando únicamente el teclado (Tab, Shift+Tab, Enter, Espacio): el foco debe ser claramente visible.
4. Aplica zoom al 200% y confirma la legibilidad de todos los elementos visuales.

21. PRUEBAS DE ERRORES Y RECUPERACIÓN

Provoca fallos controlados en el entorno aislado (ej. detener el servicio MariaDB o revocar permisos en la carpeta de subidas). **Comportamiento esperado:** La aplicación debe mostrar una pantalla de error genérica sin revelar trazas de código PHP, consultas SQL ni credenciales. Al restaurar el servicio, el sistema debe operar normalmente.

22. RENDIMIENTO Y VOLUMEN

Ejecuta mediciones de tiempo de respuesta en consultas de matching y listados con colecciones sintéticas cercanas al límite de 500 registros. Identifica y registra posibles cuellos de botella en la ejecución de consultas encadenadas.

23. PRUEBAS DE MIGRACIÓN, BACKUP Y RECUPERACIÓN

1. **Migración limpia:** Despliega un esquema desde cero siguiendo la secuencia indicada en la Sección 4 y ejecuta la suite `tests/run.php`.
2. **Backup y restauración:** Exporta la base de datos y la carpeta de convenios de prueba, restáuralos en un entorno paralelo y verifica la integridad de las descargas.

24. REGISTRO DE EJECUCIÓN Y PLANTILLA DE DEFECTOS

Para cada caso de prueba ejecutado, documenta los hallazgos en la siguiente estructura oficial:

Campo	Descripción y Contenido Requerido
ID & Entorno	Código del caso (ej. RT-01), Commit/Versión, SO, Navegador, Base `fct_test_*`.
Precondiciones	Rol activo, IDs de prueba sintéticos utilizados y estado inicial del sistema.
Resultado & Evidencia	Estado (Aprobado / Fallido / Bloqueado), Código HTTP, Captura o Log sanitizado.

25. CRITERIOS DE ACEPTACIÓN DEL CICLO DE PRUEBAS

Un ciclo de pruebas se declara oficialmente **APROBADO** únicamente cuando:

- La base aislada fue creada e inicializada sin errores de migración.
- El script `tests/run.php` devuelve el estado `OK`.
- El análisis sintáctico de PHP (Lint) pasa en el 100% de los archivos.
- Todas las comprobaciones de autorización por matriz de roles son satisfactorias.
- No existen vulnerabilidades de acceso directo a archivos PDF ni inyecciones SQL/XSS.
- Los fallos bloqueantes identificados han sido corregidos y revalidados.

26. LIMPIEZA AL TERMINAR

1. Cierra todas las sesiones activas y detén el servidor web integrado (`Ctrl + C`).
2. Elimina los archivos PDF temporales generados durante las pruebas.
3. Elimina la base de datos de pruebas `fct_test_manual`.
4. Restablece las variables de entorno de la terminal.